OpenGL Moderno:
VAO, VBO, EBO e GLSL

Eduardo R. Rizzotto
Murilo F. Soranzo
Ricardo Majolo

CONTEXTUALIZAÇÃO E FUNDAMENTOS

O que é o OpenGL Moderno e sua Finalidade

O que é o Tema e Contexto

• **Histórico:** Até a versão 2.1, o OpenGL operava em Modo Imediato (Fixed-Function Pipeline), baseado em comandos como glBegin() e glEnd().

• **OpenGL Moderno:** A partir da versão 3.3 Core Profile, o modo legado foi descontinuado e substituído pelo Pipeline Programável.

• **Finalidade Central:** Alocar dados de geometria permanentemente na memória da placa de vídeo (VRAM) e programar diretamente o processamento de vértices e pixels na GPU.

• **Relação com CG:** É a base das engines atuais (Godot, Unity, Unreal) e de APIs modernas como Vulkan, WebGPU e DirectX 12.

Problemas que Resolve

• **Gargalo CPU → GPU (Barramento):** No modo imediato, a CPU enviava coordenada por coordenada (glVertex3f) para a GPU a cada frame (60x/s), travando o barramento PCI-Express.

• **Redundância e Desperdício de VRAM:** Para desenhar 1 cubo (12 triângulos), eram enviados 36 vértices repetidos, embora o cubo possua apenas 8 cantos físicos.

• **Falta de Flexibilidade Visual:** Iluminação e projeções eram limitadas a fórmulas prontas do driver. Não era possível criar efeitos personalizados por pixel.

CONTEXTUALIZAÇÃO E FUNDAMENTOS

Os 4 Pilares Fundamentais: VBO, EBO, VAO e GLSL

1. VBO (Vertex Buffer)

Array bruto de floats alocado diretamente na VRAM.
Guarda posições (X,Y,Z), cores (R,G,B) e normais (Nx,Ny,Nz). Enviado uma única vez pela CPU.

2. EBO (Element Buffer)

Buffer de índices (uint32) na GPU que dita a conectividade. Diz quais vértices do VBO formam cada triângulo. Elimina duplicação de dados na memória.

3. VAO (Vertex Array)

O objeto gerenciador de estado. Grava a organização da memória do VBO (stride de 36B e offsets) e guarda o vínculo com o EBO ativo. Ativa tudo em 1 comando.

4. GLSL (Shaders)

Linguagem em C compilada para a GPU. Divide-se em Vertex Shader (transformações e deformação de vértices) e Fragment Shader (cálculo de cor e luz por pixel).

Possibilidades & Limitações

- Aplicações: Jogos 3D, CAD/BIM, simuladores, visualização científica (VisPy).
- Vantagem: Renderiza milhões de triângulos a 60+ FPS com controle total. Limitação: Código inicial mais verboso que o glBegin().

FUNCIONAMENTO E COMPUTAÇÃO GRÁFICA

Arquitetura do Pipeline Gráfico: CPU vs. GPU

1. CPU (Python)
Cria VBO/EBO e Matrizes 4x4

2. VBO + EBO
Dados residentes na VRAM da GPU

3. Vertex Shader
Multiplica MVP Deformação GPU

4. Rasterização
Monta triângulos e interpola dados

5. Fragment Shader
Calcula luz Phong e cor por pixel

6. Framebuffer
Z-Buffer e exibição na tela

Papel da CPU (Python)

• **Inicialização:** Envia arrays NumPy de vértices e índices para a VRAM via glBufferData() uma única vez.

• **Laço Principal:** Captura teclado, atualiza o relógio e monta apenas 3 matrizes 4x4 (Model, View, Projection).

• **Despacho Leve:** Envia as matrizes via glUniform*() e emite 1 chamada única: glDrawElements(). Não reenvia vértices!

Papel da GPU (GLSL + VRAM)

• **VRAM Dedicada:** Armazena toda a geometria pesada localmente, operando na largura de banda interna de centenas de GB/s.

• **Vertex Shader (Paralelo):** Processa milhares de vértices simultaneamente aplicando a transformação Projetiva: v_clip = P * V * M * v_local.

• **Rasterizador & Fragment Shader:** Executa interpolação baricêntrica de normais e cores e calcula a iluminação de Phong para milhões de fragmentos/pixels.

FUNCIONAMENTO E COMPUTAÇÃO GRÁFICA

Layout de Memória: Stride, Offsets e Economia com EBO

Estrutura do VBO na GPU: Stride de 36 Bytes (9 floats por vértice)

[X, Y, Z (Posição - 12B)] → Offset = 0 bytes | Location = 0 no Vertex Shader
[R, G, B (Cor - 12B)] → Offset = 12 bytes | Location = 1 no Vertex Shader
[Nx, Ny, Nz (Normal - 12B)] → Offset = 24 bytes | Location = 2 no Vertex Shader

Demonstração Prática do EBO (Cubo 3D)

- **Sem EBO (Modo das Aulas):** 6 faces × 2 triângulos × 3 vértices = 36 vértices enviados no VBO (1296 bytes).
- **Com EBO (OpenGL Moderno):** Apenas 8 vértices únicos no VBO (288 bytes) + 36 inteiros no EBO (144 bytes).
- **Economia Real:** Redução massiva de memória! Em malhas densas (terrenos, esferas), cada vértice é reutilizado por até 6 triângulos.

O Papel do VAO (O Gravador)

- **Gravador de Configuração:** Ao fazer glBindVertexArray(vao), o OpenGL passa a registrar tudo: os ponteiros de atributo e o vínculo com o EBO.
- **Troca Rápida de Objeto:** Para trocar entre o Cubo, o Octaedro e a Superfície na tela, o código chama apenas 1 linha: glBindVertexArray(novo_vao).
- **Desenho Instantâneo:** glDrawElements() lê os índices diretamente do EBO que ficou guardado dentro daquele VAO!

FUNCIONAMENTO E COMPUTAÇÃO GRÁFICA

Relação com a Disciplina:

Conceito de CG	OpenGL	OpenGL Moderno	O que mudou?
Envio de Vértices	glBegin() ... glVertex3f() ... glEnd()	VBO (glBufferData) + VAO	Substituído (Dados ficam na VRAM)
Topologia / Malhas	GL_QUADS, GL_LINE_STRIP, etc.	Apenas GL_TRIANGLES indexados por EBO	Desabstraído (Padrão das GPUs atuais)
Transformações 3D	glTranslate(), glRotatef(), glPushMatrix()	Matrizes 4x4 em NumPy (u_model)	Calculado explicitamente na CPU/GPU
Câmera e Projeção	gluPerspective(), gluLookAt(), glOrtho()	Matrizes u_view e u_projection (GLSL)	Matemática projetiva no Vertex Shader
Iluminação & Cores	glColor3f(), glLightfv(GL_LIGHT0, ...)	Modelo de Phong no Fragment Shader	Totalmente programável por pixel

CÓDIGO, DEMONSTRAÇÃO E CONCLUSÕES

Estrutura Geral da Implementação do Projeto

geometria.py

Geração de Malhas Indexadas

Define os arrays NumPy float32 para o VBO e uint32 para o EBO de 3 malhas:
- Cubo 3D (8 vértices únicos / 36 índices)
- Octaedro (6 vértices / 24 índices)
- Superfície 16x16 (terreno deformável)

transformacoes.py

Matemática 4x4 Explícita

Implementa as matrizes de CG sem usar a biblioteca GLU :
- Translação, Escala e Rotação (X, Y, Z)
- Matriz de Visão Câmera (LookAt)
- Matriz de Projeção Perspectiva (Frustum)

main.py

Pipeline e Aplicação GLFW

Gerencia o ciclo de vida completo do OpenGL 3.3 Core Profile:
- Compilação e linkagem dos Shaders
- Criação de VAO, VBO e EBO
- Loop de renderização e controles ao vivo

CÓDIGO, DEMONSTRAÇÃO E CONCLUSÕES

Trechos Principais: Alocação de Buffers e Shaders

1. Criação de VAO, VBO e EBO (main.py)

```
glBindVertexArray(vao) # 1. Ativa o gravador

# 2. VBO: Envia vértices para a VRAM da GPU
glBindBuffer(GL_ARRAY_BUFFER, vbo)
glBufferData(GL_ARRAY_BUFFER, vertices.nbytes,
        vertices, GL_STATIC_DRAW)

# 3. EBO: Envia índices dos triângulos
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, ebo)
glBufferData(GL_ELEMENT_ARRAY_BUFFER, indices.nbytes,
        indices, GL_STATIC_DRAW)

# 4. Define Stride (36B) e Offsets (0B, 12B, 24B)
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 36, 0)
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 36, 12)
glVertexAttribPointer(2, 3, GL_FLOAT, GL_FALSE, 36, 24)
```

2. Shaders GLSL e Desenho via EBO

```
// Vertex Shader: Transformação MVP na GPU
gl_Position = u_projection * u_view * u_model * vec4(pos,
1.0);

// Fragment Shader: Iluminação de Phong
vec3 ambient = u_ambient * vColor;
float diff = max(dot(norm, lightDir), 0.0);
vec3 diffuse = diff * vColor;
vec3 specular = 0.6 * spec * vec3(1.0);
FragColor = vec4(ambient + diffuse + specular, 1.0);

# No Loop da CPU: Apenas 1 comando desenha tudo!
glBindVertexArray(vao)
glDrawElements(GL_TRIANGLES, qtd_indices,
        GL_UNSIGNED_INT, None)
```

Demonstração Prática

Passo 1: Desenhando via EBO (+ / -)

Apertar 'W' (Wireframe) e usar '-' para diminuir os triângulos até sobrar 1. Em seguida, usar '+' para construir o cubo triângulo por triângulo ao vivo para a turma ver o EBO funcionando!

Passo 2: Alternância de Shaders (M)

Alternar ao vivo os 4 modos do Fragment Shader:
0 = Cores puras interpoladas do VBO
1 = Iluminação Phong (Ambiente + Difusa + Especular)
2 = Normais mapeadas em RGB
3 = Shader procedural pulsante

Passo 3: Deformação na GPU (3 + D)

Mudar para a Superfície 16x16 (tecla 3) e ligar a Deformação por Onda Senoidal (tecla D). Provar que a GPU deforma a malha via GLSL sem a CPU precisar reenviar o VBO!

Controles de Câmera e Projeção

- Setas e Espaço: Rotação nos eixos X/Y e pausa (u_model). | Z / X: Zoom da câmera LookAt (u_view).
- F / V: Ajuste contínuo do FOV da Projeção Perspectiva de 20° a 90° (u_projection). | R: Reset completo.

Conclusões Finais

Eficiência de Barramento: O trio VBO + EBO + VAO resolve definitivamente o maior gargalo da Computação Gráfica: a transferência redundante de dados entre CPU e GPU.

Otimização Geométrica (EBO): Malhas 3D modernas dependem de indexação para evitar cópias desnecessárias de atributos e economizar memória na placa de vídeo.

Poder do Pipeline Programável (GLSL): A GPU deixa de ser um executor de rotinas fixas e se torna um supercomputador paralelo programável para deformações e modelos foto-realistas de luz.

Base da Indústria Atual: Tudo o que foi implementado neste trabalho é a fundação direta de APIs contemporâneas como Vulkan, WebGL, WebGPU, DirectX 12 e Metal.